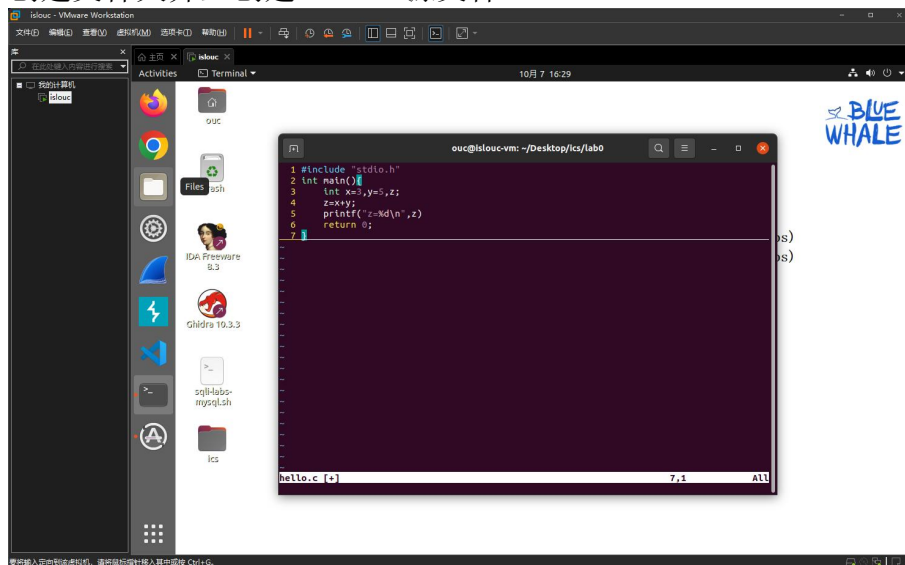


实 验 报 告

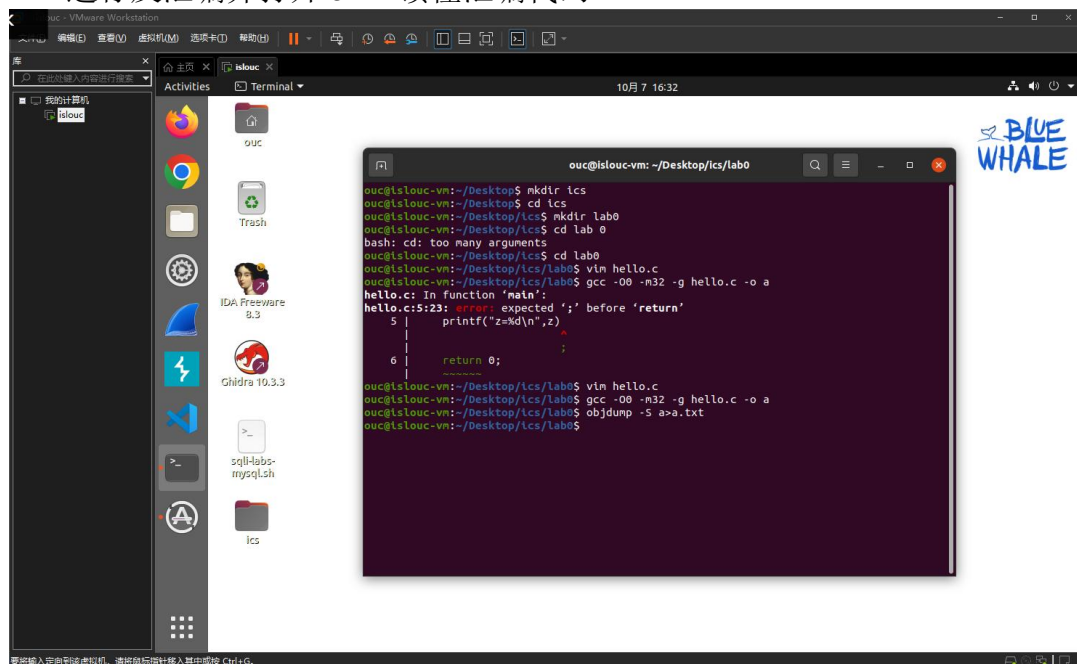
学 号	2407000 1071	姓 名	刘和雄	专业班级	2024 软件工程
课程名称	计算机系统基础 I			学期	2025 秋季学期
任课教师	李晓慧		完成日期		2025.10.7
小组成员		刘和雄			

一、实验内容及步骤（55%）

- 创建文件夹并且创建 hello.c 源文件



- 进行反汇编并打开 a.txt 读懂汇编代码



```

181 int main(){
182     11cd: f3 0f 1e fb    endbr32
183     11d1: 8d 4c 24 04    lea    0x4(%esp),%ecx
184     11d5: 83 e4 f0      and    $0xfffffff0,%esp
185     11d8: ff 71 fc      pushl  -0x4(%ecx)
186     11db: 55           push   %ebp
187     11dc: 89 e5        mov    %esp,%ebp
188     11de: 53           push   %ebx
189     11df: 51           push   %ecx
190     11e0: 83 ec 10     sub    $0x10,%esp
191     11e3: e8 44 00 00 00 call    122c <_x86_get_pc_thunk.ax>
192     11e8: 05 f0 2d 00 00 add    $0x2df0,%eax
193     11ed: c7 45 ec 03 00 00 00 movl    $0x3,-0x14(%ebp)
194     11f4: c7 45 f0 05 00 00 00 movl    $0x5,-0x10(%ebp)
195     11f9: 74 05       je     1203;
196     11fb: 8b 4d ec     mov    -0x14(%ebp),%ecx
197     11fe: 8b 55 f0     mov    -0x10(%ebp),%edx
198     1201: 01 ca       add    %ecx,%edx
199     1203: 89 55 f4     mov    %edx,-0xc(%ebp)
200     1206: 83 ec 08     sub    $0x8,%esp
201     1209: ff 75 f4     pushl  -0xc(%ebp)
202     120c: 8d 90 30 e0 ff ff lea    -0x1f0(%eax),%edx
203     1212: 52         push   %edx
204     1213: 89 c3       mov    %eax,%ebx
205     1215: e8 56 fe ff ff call    1070 <printf@plt>
206     121a: 83 c4 10     add    $0x10,%esp
207     121d: b8 00 00 00 00 mov    $0x0,%eax
208     1222: 59         pop    %ecx
209     1225: 5b         pop    %ebx
210     1227: 5d         pop    %ebp
211     1228: 8d 61 fc     lea    -0x4(%ecx),%esp
212     122b: c3         ret

```

1.变量 x,y 的初始化:

194: movl \$0x3,-0x14(%ebp); 把立即数 3 存到 [ebp-20] 的内存位置
 195: movl \$0x5,-0x10(%ebp); 把立即数 5 存到 [ebp-16] 的内存位置

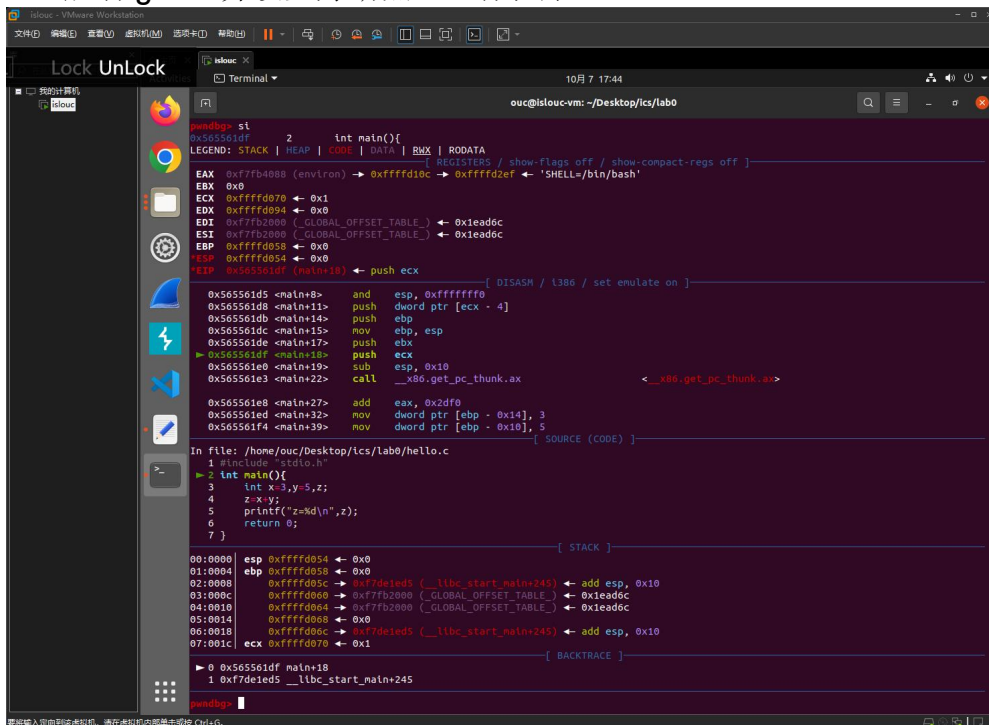
2.z=x+y 的计算:

197: mov -0x14(%ebp),%ecx; 第 1 步: 把 x 的值 (从[ebp-20]) 取出来, 放到 ecx 寄存器
 198: mov -0x10(%ebp),%edx; 第 2 步: 把 y 的值 (从[ebp-16]) 取出来, 放到 edx 寄存器
 199: add %ecx,%edx; 第 3 步: 计算 $edx = edx + ecx$ (结果是 8, 现在在 edx 里)
 200: mov %edx,-0xc(%ebp); 第 4 步: 把结果 (在 edx 里) 存到为 z 分配的内存地址 [ebp-12]

3. printf 的运行:

203: pushl -0xc(%ebp); 准备第 2 个参数: 把变量 z 的值 (8) 从内存中取出来, 压入栈中
 ...
 206: push %edx; 准备第 1 个参数: 把字符串"z=%d\n"的地址压入栈中 (地址在 edx 里)
 207: call 1070 <printf@plt>; 正式调用 printf 函数
 208: add \$0x10,%esp; 调用结束后, 清理刚才压入栈的参数, 恢复栈的平衡

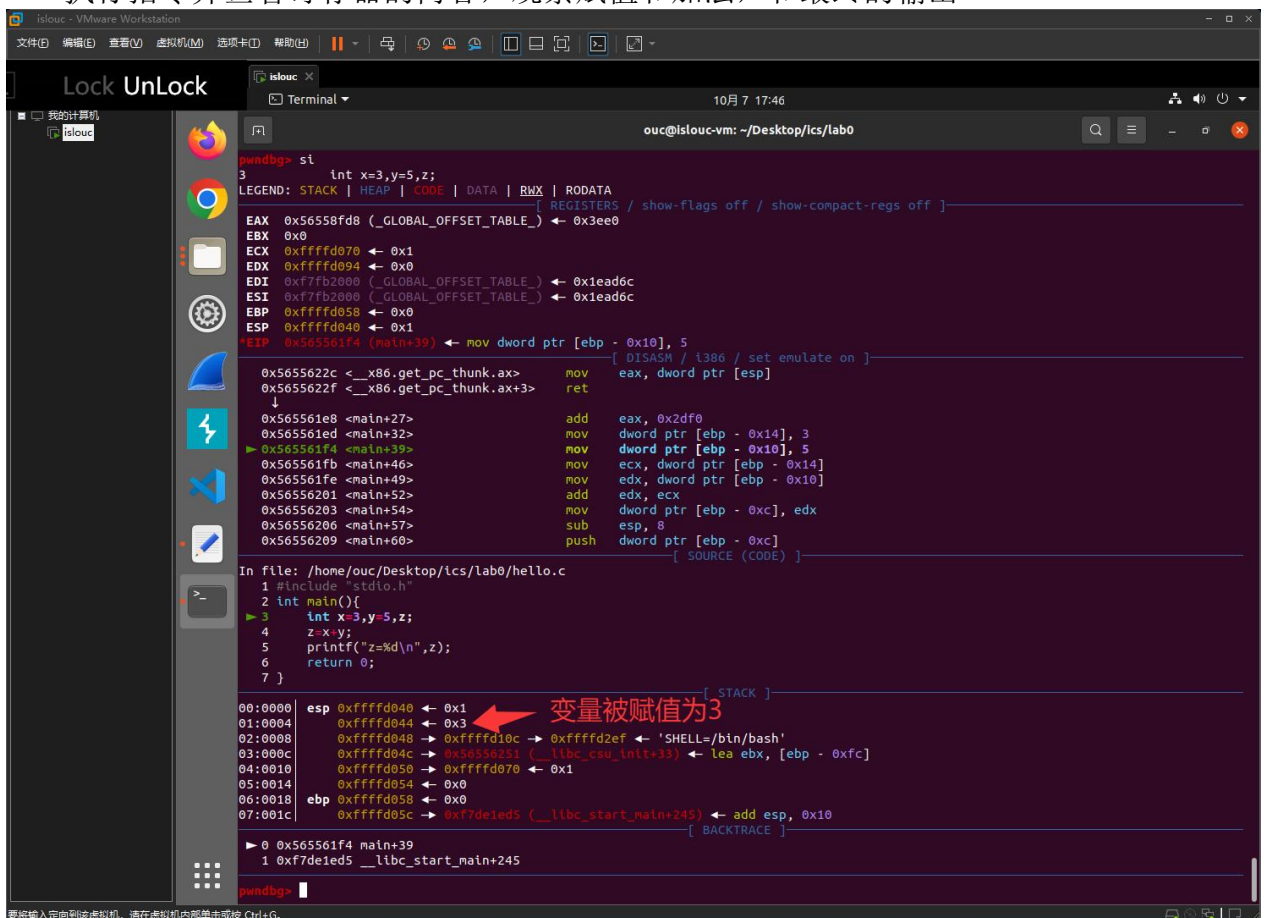
● 启动 gdb, 并设置好断点, 运行程序



```

pwndbg> st
0x565561df 2 int main(){
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[REGISTERS / show-flags off / show-compact-regs off]
EAX 0x77fb408b (environ) → 0xffffd10c → 0xffffd2ef ← 'SHELL=/bin/bash'
EBX 0x0
ECX 0xffffd070 ← 0x1
EDX 0xffffd094 ← 0x0
EDI 0x77fb2008 (_GLOBAL_OFFSET_TABLE_) ← 0x1ead6c
ESI 0x77fb2008 (_GLOBAL_OFFSET_TABLE_) ← 0x1ead6c
EBP 0xffffd058 ← 0x0
ESP 0xffffd054 ← 0x0
EIP 0x565561df (main+8) ← push ecx
[DISASM / i386 / set emulate on]
0x565561d5 <main+8> and esp, 0xfffff000
0x565561d8 <main+11> push dword ptr [ecx + 4]
0x565561db <main+14> push ebp
0x565561de <main+17> push ebx
0x565561e1 <main+1a> push ecx
0x565561e4 <main+1d> sub esp, 0x10
0x565561e7 <main+20> call __x86.get_pc_thunk.ax ← __x86.get_pc_thunk.ax>
0x565561ea <main+23> add eax, 0x2df0
0x565561ed <main+26> mov dword ptr [ebp - 0x14], 3
0x565561f0 <main+29> mov dword ptr [ebp - 0x10], 5
[BACKTRACE]
#0 0x565561df main+8
#1 0x77de1d55 __libc_start_main+245
pwndbg>
  
```

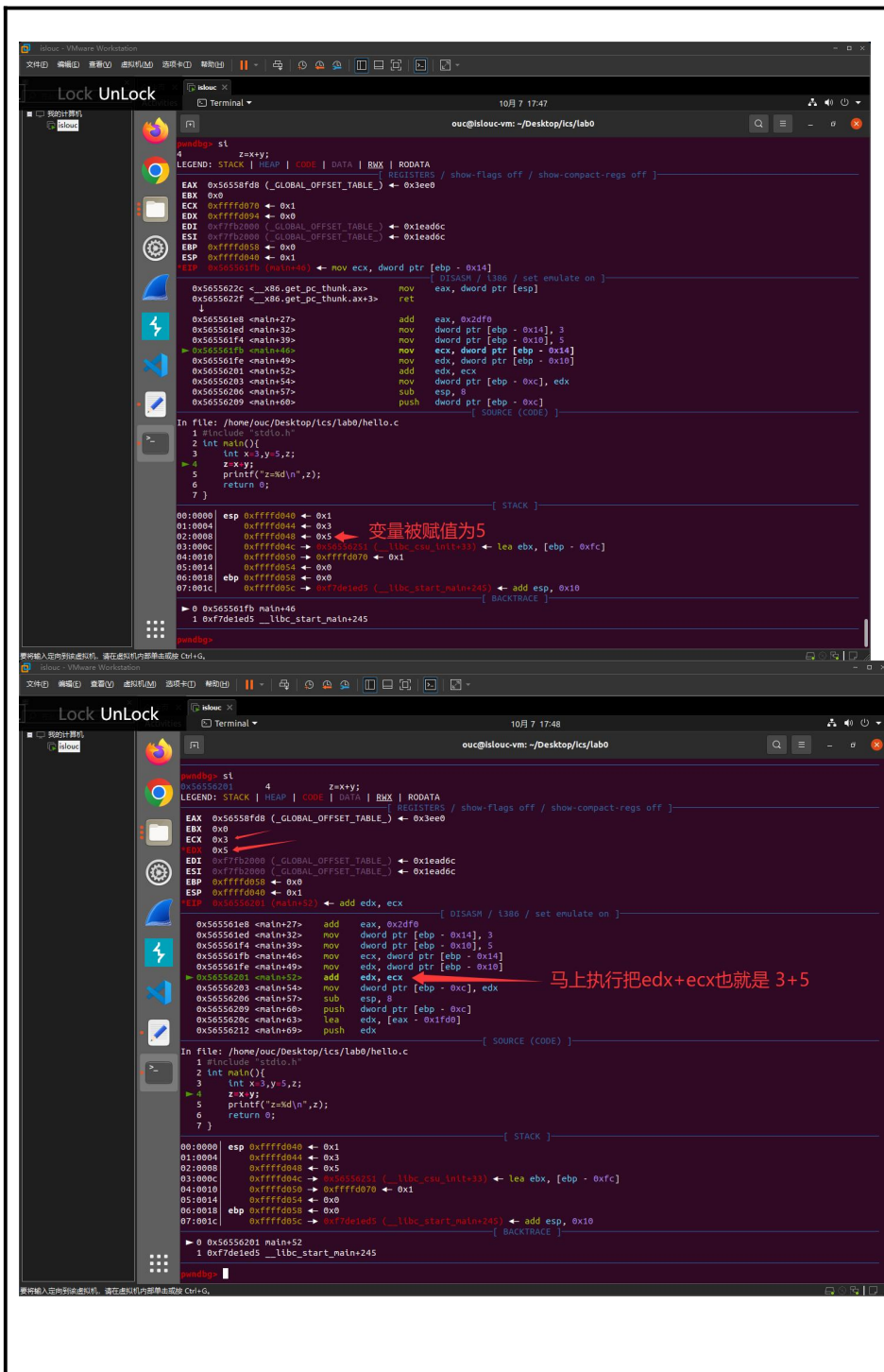
● 执行指令并查看寄存器的内容, 观察赋值和加法, 和最终的输出

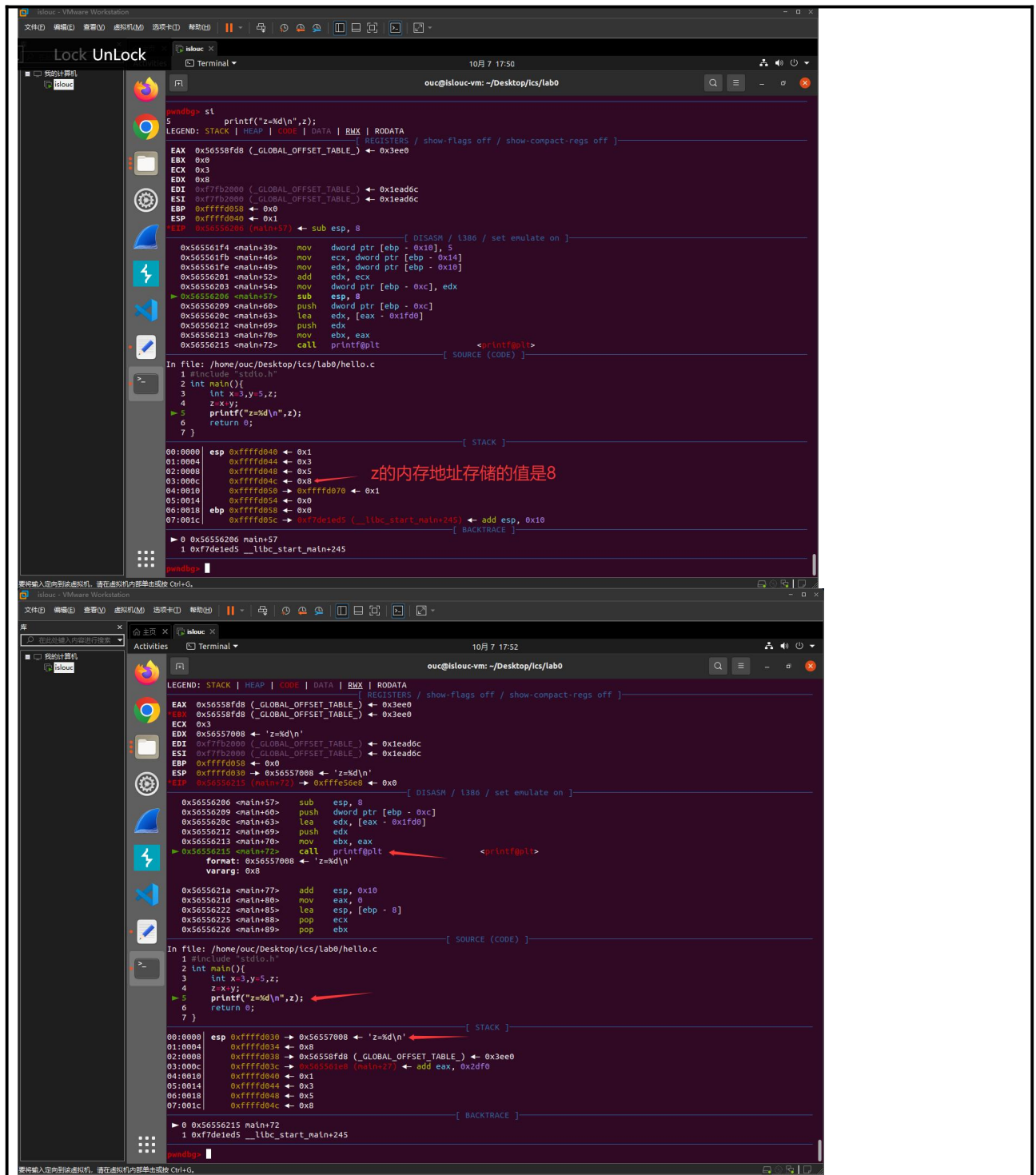


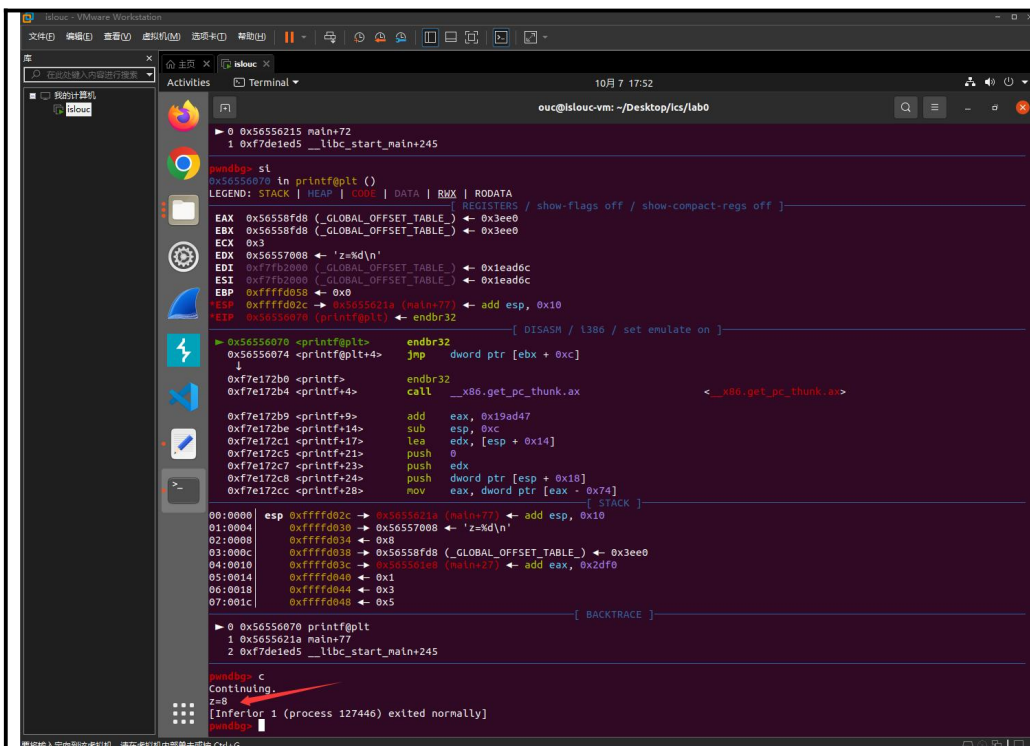
```

pwndbg> st
3 int x=3,y=5,z;
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[REGISTERS / show-flags off / show-compact-regs off]
EAX 0x565561d8 (_GLOBAL_OFFSET_TABLE_) ← 0x3ee0
EBX 0x0
ECX 0xffffd070 ← 0x1
EDX 0xffffd094 ← 0x0
EDI 0x77fb2008 (_GLOBAL_OFFSET_TABLE_) ← 0x1ead6c
ESI 0x77fb2008 (_GLOBAL_OFFSET_TABLE_) ← 0x1ead6c
EBP 0xffffd058 ← 0x0
ESP 0xffffd054 ← 0x0
EIP 0x565561f4 (main+39) ← mov dword ptr [ebp - 0x10], 5
[DISASM / i386 / set emulate on]
0x5655622c <main+39> mov eax, dword ptr [esp]
0x5655622f <main+3c> ret
[BACKTRACE]
#0 0x565561f4 main+39
#1 0x77de1d55 __libc_start_main+245
pwndbg>
  
```

变量被赋值为3







- 地址重定位，对比两者，可以发现目标文件中的地址是未定位的相对地址，而可执行文件中的地址是链接器重定位后的虚拟地址。

179 000011cd <main>:	167 0040 c10c0100 41c341c5 430c0404 10000000A.A.C.....
180 #include "stdio.h"	168 0050 50000000 00000000 04000000 00000000 P.....
181 int main()	169
182 {	170 Disassembly of section .text:
183 11cd: f3 0f 1e fb	171
184 11d1: 8d 4c 24 04	172 00000000 <main>:
185 11d5: 83 e4 f0	173 0: f3 0f 1e fb endbr32
186 11d8: ff 71 fc	174 4: 8d 4c 24 04 lea 0x4(%esp),%ecx
187 11db: 55	175 8: 83 e4 f0 and \$0xffffffff0,%esp
188 11dc: 89 e5	176 b: ff 71 fc pushl -0x4(%ecx)
189 11de: 53	177 e: 55 push %ebp
190 11df: 51	178 f: 89 e5 mov %esp,%ebp
191 11e0: 83 ec 10	179 11: 53 push %ebx
192 11e3: e8 44 00 00 00	180 12: 51 push %ecx
193 int x=3,y=5,z;	181 13: 83 ec 10 sub \$0x10,%esp
194 11ed: c7 45 ec 03 00	182 16: e8 fc ff ff ff call 17 <main+0x17>
195 11f4: c7 45 f0 05 00	183 1b: 05 01 00 00 00 add \$0x1,%eax
196 z=x+y;	184 20: c7 45 ec 03 00 00 00 movl \$0x3,-0x14(%ebp)
197 11fb: 8b 4d ec	185 27: c7 45 f0 05 00 00 00 movl \$0x5,-0x10(%ebp)

二、实验结果（35%）

- 本次实验通过对一个简单的C语言程序 hello.c 进行编译、反汇编和动态调试，成功地观察并分析了程序从高级语言到机器指令的转换过程，以及在计算机内部的实际执行流程。
- 本次实验成功地将C语言代码、底层汇编指令和实际的机器执行过程三者有机地联系了起来。静态分析揭示了程序的理论结构，而动态调试则提供了无可辩驳的运行证据。通过本次实践，我深刻理解了局部变量的存储方式、基本运算的实现原理以及函数调用的底层机制，对程序的机器码表示有了更为具体和深入的认识。

- 总而言之，本次实验不仅让我掌握了 `gcc`, `objdump`, `gdb` 等核心工具的使用，更重要的是，它引导我从“代码编写者”的视角，深入到“机器执行者”的视角，建立起了从高级语言到机器代码的思维桥梁。这种自底向上的思考方式，对于未来编写更高效、更健壮的程序，以及排查疑难问题都将大有益处。